Київський національний університет будівництва і архітектури

Кафедра Кібербезпеки та комп’ютерної інженерії

МАГІСТРАТУРА

Лабораторна робота №3

Тема: Розробка графічного інтерфейсу керування роботом (Qt + ROS2)

Виконав студент КСМм-26:
Павлюк О.А.

Перевірив
Маринський К.В.

Київ – 2026

Мета:

Сформувати практичні навички створення графічного інтерфейсу користувача (GUI) у ROS2, навчитися інтегрувати GUI з нодами ROS2, реалізувати передачу команд руху через Topic /cmd_vel, а також організувати відображення статусу системи.

Завдання:

1. Підготовка середовища
1.1. Активувати ROS2-середовище.
1.2. Перевірити працездатність команд:
• ros2 node list
• ros2 topic list
1.3. Запустити Motion Node з лабораторної №2.
2. Створення пакету GUI
2.1. Створити новий пакет типу ament_python для реалізації GUI-ноди.
2.2. Додати залежності:
• rclpy
• geometry_msgs
2.3. Зібрати workspace та перевірити коректність встановлення пакету.
3. Розробка GUI
3.1. Створити графічне вікно з кнопками:
• Вперед
• Назад
• Ліворуч
• Праворуч
• Стоп
3.2. Додати текстове поле для відображення статусу.
4. Інтеграція GUI з ROS2
4.1. Реалізувати Publisher на Topic:
/cmd_vel
4.2. Використати тип повідомлення:
geometry_msgs/msg/Twist
4.3. Реалізувати передачу команд при натисканні кнопок:
• linear.x — рух вперед/назад
• angular.z — поворот
5. Відображення статусу
5.1. Реалізувати підписку на Topic статусу (наприклад /motion_status).
5.2. Виводити отримані повідомлення у GUI.
6. Тестування
6.1. Запустити GUI-ноду.

6.2. Запустити Motion Node.
6.3. Перевірити керування роботом через GUI.
6.4. Переконатися, що команди передаються коректно.

Звіт

1) Опис реалізації GUI-ноди

Код наведений в додатку А

GUI-нода включає:

1. Паблішер в топік /cmd_vel для задання швидкостей та /motion_status для виводу повідомлень в графічний інтерфес.
2. Підписка на топік статусу /motion_status.
3. Встановлення значень швидкостей для кнопок self.linear_speed та self. angular_speed.
4. Створення віджетів (кнопок та поля повідомлень).

Фрагменти коду:

```
# Publisher на /cmd_vel для керування
self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)

# Publisher на /motion_status для публікації статусу кнопок
self.status_publisher = self.create_publisher(String, '/motion_status', 10)

# Підписка на топік статусу
self.subscription = self.create_subscription(
    String,
    '/motion_status',
    self.status_callback,
    10
)
```

```python
def create_widgets(self):
    # Заголовок
    title_label = tk.Label(self.root, text="Керування роботом", font=("Arial", 14, "bold"), bg="#f0f0f0")
    title_label.pack(pady=10)

    # Кнопки керування із прив'язкою назв до /motion_status
    btn_frame = tk.Frame(self.root, bg="#f0f0f0")
    btn_frame.pack(pady=10)

    self.btn_forward = tk.Button(btn_frame, text="▲ Вперед", width=12, height=2, bg="#4CAF50", fg="white", font=("Arial", 10, "bold"),
                                 command=lambda: self.ros_node.handle_button_press(self.ros_node.linear_speed, 0.0, "Вперед"))
    self.btn_forward.grid(row=0, column=1, padx=5, pady=5)

    self.btn_left = tk.Button(btn_frame, text="◄ Ліворуч", width=12, height=2, bg="#2196F3", fg="white", font=("Arial", 10, "bold"),
                              command=lambda: self.ros_node.handle_button_press(0.0, self.ros_node.angular_speed, "Ліворуч"))
    self.btn_left.grid(row=1, column=0, padx=5, pady=5)

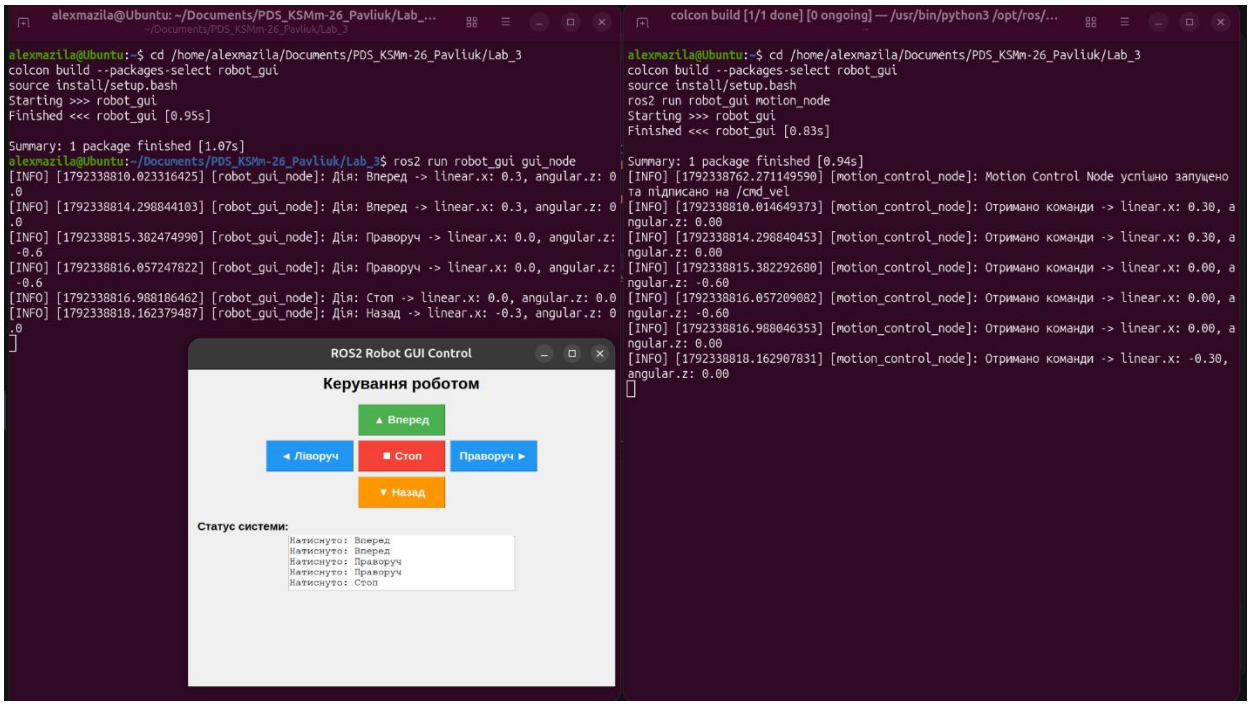
    self.btn_stop = tk.Button(btn_frame, text="■ Стоп", width=12, height=2, bg="#f44336", fg="white", font=("Arial", 10, "bold"),
                              command=lambda: self.ros_node.handle_button_press(0.0, 0.0, "Стоп"))
    self.btn_stop.grid(row=1, column=1, padx=5, pady=5)

    self.btn_right = tk.Button(btn_frame, text="Праворуч ►", width=12, height=2, bg="#2196F3", fg="white", font=("Arial", 10, "bold"),
                               command=lambda: self.ros_node.handle_button_press(0.0, -self.ros_node.angular_speed, "Праворуч"))
    self.btn_right.grid(row=1, column=2, padx=5, pady=5)

    self.btn_backward = tk.Button(btn_frame, text="▼ Назад", width=12, height=2, bg="#FF9800", fg="white", font=("Arial", 10, "bold"),
                                  command=lambda: self.ros_node.handle_button_press(-self.ros_node.linear_speed, 0.0, "Назад"))
    self.btn_backward.grid(row=2, column=1, padx=5, pady=5)
```

2) Тестуємо ноду.

Скриншот роботи інтерфесу та нод (gui та motion):



3) Опис взаємодії: GUI → ROS2 → робот

Користувач натискає кнопки управління, ROS2 отримує відповідні значення лінійної та кутової швидкості і публікує їх у топік /cmd_vel.

Нода Motion підписана на даний топік і отримує значення швидкості, перевіряє чи допустимі вони по граничним значенням, що ми задавали в минулій роботі №2. З реалізованим зв’язком з роботом ця нода давала б сигнал для коліс робота.

Також публікується натиснута кнопка в графічний інтерфейс як перевірка правильності обробки сигналів від користувача.

4) Висновки

На цій лабораторній роботі ми розробили графічний інтерфейс керування роботом.

На основі проведеного тестування у симуляторі було виявлено зручності GUI порівняно з консольним керуванням:

1. GUI забезпечує інтуїтивно зрозуміле керування за допомогою звичних кнопок. Оператору не потрібно запам'ятовувати та набирати команди.
2. Наявність окремого текстового поля для статусу дозволяє в режимі реального часу бачити зворотний зв'язок від системи.
3. Графічний інтерфейс на базі Qt легко розширювати: у майбутньому до нього можна додати слайдери регулювання швидкості, поля виведення даних з камери або аварійні індикатори.

Матеріали роботи було структуровано та опубліковано на GitHub https://github.com/alexmazila/PDS_KSMm-26_Pavliuk/tree/main

Додаток А

```
#!/usr/bin/env python3
import sys
import threading
import tkinter as tk

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from std_msgs.msg import String


class RobotGuiNode(Node):
    def __init__(self):
        super().__init__('robot_gui_node')

        # Publisher на /cmd_vel для керування
        self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)

        # Publisher на /motion_status для публікації статусу кнопок
        self.status_publisher = self.create_publisher(String, '/motion_status', 10)

        # Підписка на топік статусу
        self.subscription = self.create_subscription(
            String,
            '/motion_status',
            self.status_callback,
            10
        )

        # Налаштування швидкостей для кнопок
        self.linear_speed = 0.3
        self.angular_speed = 0.6

    def handle_button_press(self, linear_x, angular_z, button_name):
        # Публікуємо швидкість у /cmd_vel
        twist_msg = Twist()
        twist_msg.linear.x = float(linear_x)
        twist_msg.angular.z = float(angular_z)
        self.publisher_.publish(twist_msg)
```

```
        # Публікуємо назву кнопки у /motion_status
        status_msg = String()
        status_msg.data = f"Натиснуто: {button_name}"
        self.status_publisher.publish(status_msg)

        self.get_logger().info(f'Дія: {button_name} -> linear.x: {linear_x}, angular.z: {angular_z}')

    def status_callback(self, msg: String):
        # Метод для передачі статусу у головне вікно GUI
        if hasattr(self, 'update_status_gui'):
            self.update_status_gui(msg.data)


class GuiApp:
    def __init__(self, root, ros_node: RobotGuiNode):
        self.root = root
        self.ros_node = ros_node
        self.root.title("ROS2 Robot GUI Control")
        self.root.geometry("350x400")
        self.root.config(bg="#f0f0f0")

        # Зв'язуємо колбек ROS-ноди з оновленням інтерфейсу
        self.ros_node.update_status_gui = self.display_status

        self.create_widgets()

    def create_widgets(self):
        # Заголовок
        title_label = tk.Label(self.root, text="Керування роботом", font=("Arial", 14, "bold"), bg="#f0f0f0")
        title_label.pack(pady=10)

        # Кнопки керування із прив'язкою назв до /motion_status
        btn_frame = tk.Frame(self.root, bg="#f0f0f0")
        btn_frame.pack(pady=10)

        self.btn_forward = tk.Button(btn_frame, text="▲ Вперед", width=12, height=2, bg="#4CAF50", fg="white", font=("Arial", 10, "bold"),
```

```
                        command=lambda:
self.ros_node.handle_button_press(self.ros_node.linear_speed, 0.0, "Вперед"))
        self.btn_forward.grid(row=0, column=1, padx=5, pady=5)

        self.btn_left = tk.Button(btn_frame, text="◄ Ліворуч", width=12, height=2,
bg="#2196F3", fg="white", font=("Arial", 10, "bold"),
                        command=lambda:          self.ros_node.handle_button_press(0.0,
self.ros_node.angular_speed, "Ліворуч"))
        self.btn_left.grid(row=1, column=0, padx=5, pady=5)

        self.btn_stop = tk.Button(btn_frame, text="□ Стоп", width=12, height=2,
bg="#f44336", fg="white", font=("Arial", 10, "bold"),
                        command=lambda: self.ros_node.handle_button_press(0.0, 0.0,
"Стоп"))
        self.btn_stop.grid(row=1, column=1, padx=5, pady=5)

        self.btn_right = tk.Button(btn_frame, text="Праворуч ►", width=12, height=2,
bg="#2196F3", fg="white", font=("Arial", 10, "bold"),
                        command=lambda: self.ros_node.handle_button_press(0.0, -
self.ros_node.angular_speed, "Праворуч"))
        self.btn_right.grid(row=1, column=2, padx=5, pady=5)

        self.btn_backward = tk.Button(btn_frame, text="▼ Назад", width=12, height=2,
bg="#FF9800", fg="white", font=("Arial", 10, "bold"),
                        command=lambda:          self.ros_node.handle_button_press(-
self.ros_node.linear_speed, 0.0, "Назад"))
        self.btn_backward.grid(row=2, column=1, padx=5, pady=5)

        # Текстове поле для відображення статусу
        status_label = tk.Label(self.root, text="Статус системи:", font=("Arial", 10,
"bold"), bg="#f0f0f0")
        status_label.pack(anchor="w", padx=20, pady=(10, 0))

        self.status_text = tk.Text(self.root, height=5, width=38, state=tk.DISABLED,
font=("Courier", 9))
        self.status_text.pack(padx=20, pady=5)

    def display_status(self, message):
        self.status_text.config(state=tk.NORMAL)
        self.status_text.insert(tk.END, f"{message}\n")
```

```
        self.status_text.see(tk.END)
        self.status_text.config(state=tk.DISABLED)


def main(args=None):
    rclpy.init(args=args)
    node = RobotGuiNode()

    # Запуск ROS2 спінінга у фоновому потоці
    ros_thread = threading.Thread(target=rclpy.spin, args=(node,), daemon=True)
    ros_thread.start()

    # Запуск графічного інтерфейсу
    root = tk.Tk()
    app = GuiApp(root, node)

    try:
        root.mainloop()
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```